Republic of the Philippines
CEBU TECHNOLOGICAL UNIVERSITY
ARGAO CAMPUS
Ed Kintanar Street, Lamacan, Argao, Cebu
Website:http://www.argao.ctu.edu.ph E-mail:cdargao@ctu.edu.ph
Phone No.: (032) 401-0737 local 1700
BAGONG PILIPINAS

w

Cross Platform Script Development
Week 3
Information Technology Department

Name:	
Course:	**BSIT**
Year & Section:	
Date:	08/20/2025
Topic:	JavaScript Designs Patterns – Singleton Design
Activity:	

Notes:

Singleton Pattern – it ensures that a class or object has **only one instance** throughout the lifetime of an application, and it provides a way to **access that single instance globally**.

Analogy:
Think of it as a **"single source of truth"** object.

Why do we need Singleton?
Without a Singleton:

- You could end up creating **multiple objects** doing the same thing → wasted memory, inconsistent behavior.
- Example: If you had **multiple database connections**, performance would drop.

With a Singleton:

- Only **one object exists** and is reused.
- Ensures **consistency** across the app.

How Singleton Works in JavaScript
Since JS is **prototype-based**, there's no built-in "private constructor" like in other OOP languages.
But we can achieve Singleton using:

- **Object literals** (simple, one object only).
- **Closures** (control instance creation).
- **Classes** with a static property (modern ES6+ approach).

Advantages

- ✓ Ensures **only one instance** exists.
- ✓ Saves **memory & resources**.
- ✓ Centralized management (all access goes through one object).
- ✓ Easy to implement in JavaScript.

Disadvantages

- Can lead to global state dependency (hard to debug if too many parts depend on it).
- Makes unit testing harder (since instance is shared).
- Violates Single Responsibility Principle if overused (object may end up doing too much).

When to Use

- ✅ Database connection pools
- ✅ Caching / configuration objects
- ✅ Logging services
- ✅ Event dispatchers
- ✅ State management

Singleton Pattern Code

```
//Note
// Singleton Pattern Example in JavaScript

class Singleton {
  constructor() {
    if (Singleton.instance) {
      return Singleton.instance; // return the same instance if already created
    }

    // properties
    this.timestamp = new Date().toISOString();

    // save the instance
    Singleton.instance = this;

    return this;
  }
```

```
  showMessage() {
    console.log("Hello! I am a Singleton instance created at:", this.timestamp);
  }
}

// Usage
const obj1 = new Singleton();
obj1.showMessage();

const obj2 = new Singleton();
obj2.showMessage();

console.log("Are obj1 and obj2 the same?", obj1 === obj2); // true ✅
```

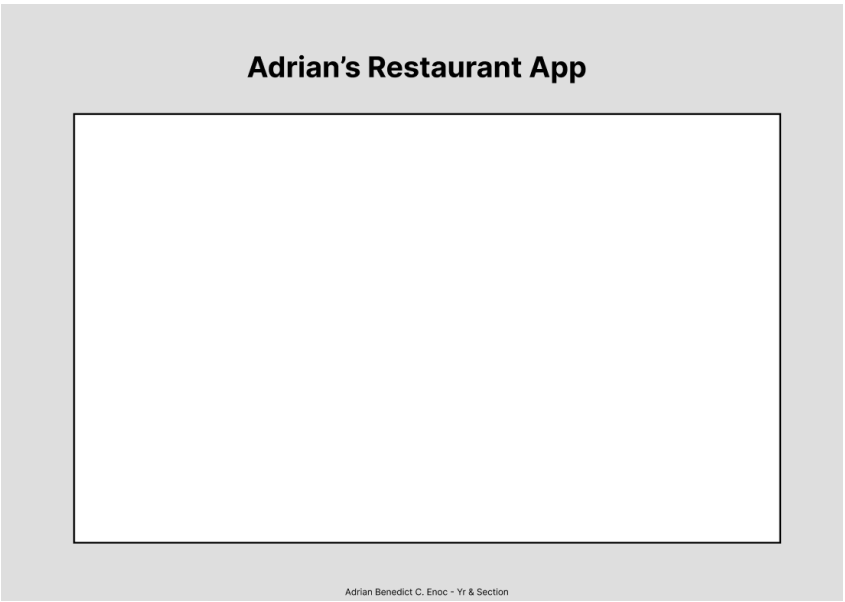
Activity: Restaurant Ordering System using Factory Pattern

Create a JavaScript mini-app that allows the user to order and display final order summary of the items selected in a HTML Page.

Requirements:

1. On title of the app it would show "YourFirstName Restaurant App"
2. On the footer or bottom of the app, it would display your full name inside an p html tag.
3. Create a restaurant factory that can handle 3 categories:
    - foodFactory (Spaghetti, Fried Chicken, Burrito, etc.)
    - drinkFactory (Water, Sprite, Ice Tea, etc.)
    - dessertFactory (Ice Cream, Cake, Donut, etc.)
4. Each item must have:
    - name
    - price
    - ingredients
    - describe() method.
5. Allows the user to select via the html page from all of the 3 categories.
6. Generate a final order summary with total price

Sample Output:



Submission of Activity

1. Upload to Google Classroom: compress your work on .zip or .7zip format only example: activity-2.zip
2. Deadline would be on **August 29 @ exactly 11:59 AM**
3. If you have questions regarding on this activity, just send a message at the google classroom.